Company Engineering

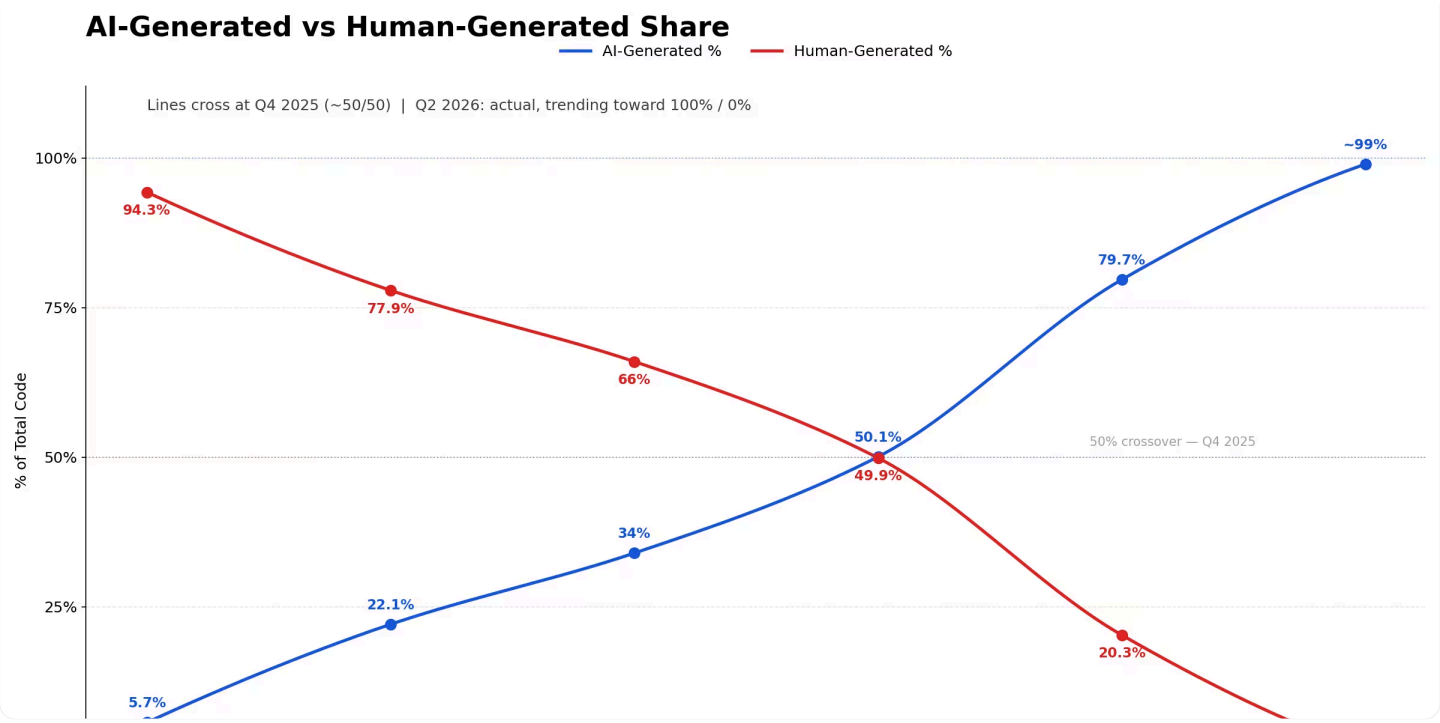
Interviewing Engineers in the AI Era: Lessons from a Year of Rebuilding

By Coinbase Jul 13, 2026

TL;DR: Now that nearly all of our new code is AI generated and overseen by humans, we've rebuilt our engineering interview loop to keep pace, testing how candidates direct AI, evaluate its output, and apply judgment where models fall short. This post documents three phases of that work: what changed, what the data showed, and what we're still figuring out.



In Q1 2025, AI-generated code represented 5.7% of everything merged into our codebase. In Q4 2025, AI crossed the 50% threshold for the first time. While humans still review all of our code, that share has reached roughly 100% without sacrificing quality or compliance guardrails. Human-authored code (not human-reviewed code) fell sharply as background agent-generated code rose.



That shift changed a lot at Coinbase. It changed how our engineers spend their time. Less writing code from scratch, more directing AI, reviewing its output, catching its architectural mistakes, and making judgment calls that models still can't make. It changed our development workflows, our deployment infrastructure, and our approach to code review.

The nature of senior engineering work has shifted fundamentally. Our engineers now spend the majority of their time authoring specs, directing AI to generate implementations, reviewing AI-produced pull requests for correctness and security, catching architectural errors that models introduce confidently, and making the judgment calls about tradeoffs, rollback risk, and system boundaries that AI still can't make reliably. Writing code from scratch is no longer where most of the value is created. As one of our engineering leaders put it: "When the cost of building goes to zero, the cost of identifying *what* to build, verifying it's correct, and getting it out safely becomes the limiting factor."

This shift to AI-generated code also forced us to confront an uncomfortable question about our hiring process: if nearly all the code we ship is AI-generated and human-reviewed, should we still ask engineering candidates to write code from scratch, from memory, under a 45-minute clock?

This post documents the answer we built over the past year: what we changed, what we learned, and what we're still figuring out.

The Challenge

Traditional engineering job interviews were designed for a world where architectural knowledge was the scarce resource. Questions like "design a URL shortener" or "build a message queue" tested whether a candidate had internalized canonical patterns like caching strategies, sharding approaches, consistency tradeoffs and if they could recall them under pressure.

That bar made sense when those skills were hard to acquire. They aren't anymore. Any competent engineer can ask an AI assistant for a reference architecture for any of those problems and receive a defensible, technically accurate answer in under a minute.

The challenge runs deeper than just the questions becoming outdated. We found our existing loop had developed two failure modes that cut in opposite directions:

False positives: Candidates who had studied hard and could recite the right patterns for URL shorteners and distributed queues would pass our process while demonstrating little of the judgment we actually need on the job. The signal we got was preparation, not capability.

False negatives: Candidates with strong architectural judgment but less pattern memorization would struggle with questions designed around recall. These were often exactly the engineers who would thrive in an AI-native environment, people who knew how to direct, evaluate, and course-correct AI output rather than reproduce textbook answers.

We also found our interview loop was redundant. An internal analysis showed an 84% correlation between two different interview rounds, meaning two expensive rounds produced nearly the same signal, and neither was measuring what we needed most.

The interview process for engineering managers hadn't been substantively updated in two years. And critically, none of our existing rounds were sufficiently focused on how a candidate actually uses AI tools, the skill that now defines a significant portion of day-to-day engineering work.

How We Rebuilt in Three Phases

We didn't redesign the interview loop all at once. We ran it like any engineering project: start with a pilot, measure against exit criteria, expand only when data supports it.

Phase 1 — H2 2025: The Frontend Pilot

The first thing we learned was that you can't simply turn AI on inside an existing interview and call it AI-native. When we enabled full AI assistance on our standard frontend coding questions, the AI solved most of them directly. The questions hadn't been designed for an AI-present environment, they had been designed to measure what a human could hold in working memory.

So, we rebuilt the questions from scratch. The new frontend interview gave candidates access to AI tools and asked them to work on realistic problems but the problems were structured so that the interesting signal came from *how* the candidate used AI: the quality of their prompts, how they evaluated the output, how they caught errors, how they iterated. Getting AI to generate a solution was easy. Generating a *good* solution, knowing when to trust it, and knowing when to push back required skills.

The frontend pilot completed successfully. Interviewer efficiency improved measurably, and early candidate feedback was positive. We had a working model to expand from.

Phase 2 — January 2026: Expanding to Backend

The expansion to backend engineering interviews required building new questions, not adapting existing ones. We developed custom repository-based questions where candidates worked in a realistic codebase: debugging issues, reviewing code for correctness and performance, reasoning about rollback scenarios, all with AI tools available.

This format tested what our engineers actually do: not writing from scratch, but working *with and against* an existing codebase and an AI collaborator to generate an improved result. Can they triage real issues? Can they catch subtle bugs that AI introduces? Do they understand what they're accepting when they accept an AI-generated change?

Phase 3 — March 2026: Company-Wide AI Fluency Launch

In March 2026, we rolled out AI signals at every stage of the engineering interview loop, not as an overlay on the existing process, but how we evaluate every candidate.

Here is what changed at each stage:

Interview Stage	Before	After
Pre-screen (coding assessment)	Standard coding assessment	AI-Assisted Coding Assessment — candidates use AI tools throughout
Live coding / Tech Execution	No AI tools permitted	AI Support & Reference enabled; candidates use documentation and AI assistance as they would at work
Domain / Reverse System Design (applicable only to senior engineering roles)	Standard system architecture discussion	Now includes questions surfacing how candidates have used AI in real systems they've built
Foundational (Hiring Manager interview)	Standard leadership questions	Added AI Competency questions and rubric; every

The AI Fluency definition we created covers three dimensions, and it applies equally to junior and senior engineers:

1. **Usage** — does the candidate use AI tools effectively and responsibly? Do they select the right tool, apply it in the right parts of their workflow, and produce measurably better outcomes?
2. **Application** — do they know *when* AI is and isn't the right solution? Can they design AI-enabled workflows that create real business impact rather than just automating tasks?
3. **Understanding Limits** — do they understand where AI breaks down? Can they identify the privacy and security implications, and apply appropriate human judgment as a guardrail?

The early signal from the new process is encouraging. While the sample size is still small, candidates who pass the AI-Assisted coding assessment are advancing through their onsite interviews at a meaningfully higher rate than candidates on the old assessment. The directional signal suggests we're identifying the right candidates to progress into the onsite, not raising a barrier. We'll share more precise data as the sample grows.

What We Measure: Three Baseline Signals

After a year of iteration, we've landed on three signals that we believe a complete engineering interview loop must produce. The specific format of each round can, and will, change as AI capabilities evolve. But these three signals are durable because they reflect what the job actually requires in today's environment.

As execution gets automated, judgment and taste are the traits that matter most. *Knowing what's actually worth building, telling a genuinely good solution from one that only looks right, and sensing when to override the model.* We don't test this in a separate round; it's what each of the signals below is ultimately reading for.

Not all of these are fully operational today. We've labeled each with its current maturity so you know exactly where we stand.

1. Repo-based coding and debugging (Live)

How does a candidate work in an existing codebase?

Can they triage issues, debug failures in code they didn't write, reason about performance and compatibility, use AI effectively on a real codebase, and show the judgment to tell a correct change from one that merely looks plausible? This is the closest analog to most

engineering work, since you almost never build on a blank slate.

2. System design with AI (Early Testing)

Can a candidate architect complex systems by directing AI to explore the design space (generating schemas, drafting API contracts, stress-testing failure modes) while regularly applying the judgment that AI cannot substitute for?

The scarce resource in system design is no longer pattern recall; it's the taste and judgment to know which constraints matter, which tradeoffs are real, and when to override what the model produces. We're actively designing this round and expect to pilot it in the coming quarters.

3. Leadership and Behavioral (Live)

This hasn't changed in its purpose: assessing how a candidate makes decisions, decides what's worth building, handles disagreement, builds trust, and grows. It has changed in one dimension: we now include direct questions about how they use AI in their day-to-day work, as concrete behavioral evidence rather than abstract capability.

One constraint we've held firm on throughout this redesign: **we don't add rounds**. Every new signal must either replace an existing round or capture something clearly distinct that the current loop doesn't measure. A longer interview is a worse candidate experience and a harder commitment to ask of interviewers. If we can't make the case for why the new round captures a different signal, we don't add it.

How we operate: the working group model. None of this came from a single top-down mandate. A cross-functional group of engineers, hiring managers, and recruiters owns the loop and keeps iterating on it: every change starts as a pilot with clear success criteria, ships only when the data supports it, and gets revisited on a regular cadence. The interview loop isn't something we refresh every few years. It's a living process that we tune as the work itself changes. That's a big part of how we moved this fast.

What this means if you're applying. If you're considering a role at Coinbase, here's what to expect: come prepared to use the tools you'd use at work with us. We're not testing memorization or pattern recall; we're evaluating how you direct AI, how you critique its output, and how you exercise judgment to determine whether and when the model gets something wrong. The best preparation is doing the kind of work you'd do on day one.

What We're Still Exploring

We want to be honest about what's unresolved, because the industry is still in early days on this.

The half-life problem. AI models are evolving fast enough that a question designed to test AI fluency today may be trivially solvable by next quarter's model. We've structured our process to account for this: the interview loop is now a living system with quarterly review cycles and an active working group responsible for monitoring question effectiveness and retiring anything that's no longer producing signal. This is a different operating model than most companies use for their interview processes, and it requires real ongoing investment.

How well does interview performance predict job performance? We're exploring how AI fluency in an interview predicts AI fluency on the job. We've built in 45- and 90-day pulse surveys for every new hire, with the goal that 100% of new hires maintain or improve their AI Fluency rating at those checkpoints and are using available AI tools measurably in their work. We'll publish what we learn.

What comes next? We're exploring a V2 of the loop that goes further: combining early rounds to reduce time-to-signal, A/B testing changes in isolation so we can attribute outcomes cleanly, and potentially building a pre-onsite signal that candidates will engage with. We're also watching closely whether the three baseline signals remain stable as the technology continues to shift.

Why We're Sharing This

We want the bar we interview against to reflect the bar we operate against. We can't hire engineers to work alongside AI if we're still selecting for the ability to work without it.

We're publishing this because everyone in the industry is navigating the same shift, and we think transparency about what worked, what didn't, and what's still unproven serves everyone better than quietly reinventing the same wheel. If you're rebuilding your engineering interview process and want to compare notes, we'd welcome the conversation. And if you're considering a job at Coinbase, we welcome you to apply.

Recent stories



Interviewing Engineers in the AI Era: Lessons from a Year of Rebuilding

Coinbase
Jul 13, 2026

TL;DR: Now that nearly all of our new code is AI generated and overseen by humans, we've rebuilt our engineering interview loop to keep pace, testing how candidates direct AI, evaluate its output, and apply judgment where models fall short. This post documents three phases of that work: what changed, what the data showed, and what we're still figuring out.

Company | Engineering

Disclaimers: Derivatives trading through the Coinbase Advanced platform is offered to eligible EEA customers by Coinbase Financial Services Europe Ltd. (CySEC License 374/19). In order to access derivatives, customers will need to pass through our standard assessment checks to determine their eligibility and suitability for this product.



Company

About

Careers

Affiliates

Blog

Press

Security

Investors

Vendors

Legal & privacy

Cookie policy

Cookie preferences

Do Not Sell or Share My Personal Information

Digital Asset Disclosures

Learn

Explore crypto

Explore stocks

Market statistics

Coinbase Bytes newsletter

Crypto basics

Tips & tutorials

Crypto glossary

Market updates

What is Bitcoin?

What is crypto?

What is a blockchain?

How to set up a crypto wallet?

How to send crypto?

Taxes

Support

Help center

Contact us

Create account

ID verification

Account information

Payment methods

Account access

Supported crypto

Status

Individuals

Buy & sell

Base App

Coinbase One

Coinbase Wealth

Credit Card

Debit Card

Explore predictions

Businesses

Asset Listings

Coinbase Business

Payments

Token Manager

Institutions

Prime

Staking

Exchange

International Exchange

Derivatives Exchange

Verified Pools

Developers

Developer Platform

Base

Server Wallets

Embedded Wallets

Base Accounts (Smart Wallets)

Onramp & Offramp

x402

Trade API

Paymaster

OnchainKit

Data API

Verifications

Node

AgentKit

Staking

Faucet

Exchange API

International Exchange API

Prime API

Derivatives API

Asset prices

Bitcoin price

Ethereum price

Solana price

XRP price

Stock prices

NVIDIA price

Apple price

Microsoft price

Amazon price

© 2026 Coinbase